Sprint 2 Report

e-Project

Team 6

Arina Agayeva, Magdalena Zheleva

Overview

During this sprint, we implemented the latest stable version of Bootstrap for the project to display correctly across devices. We also added features allowing logged-in users to add projects and allowing all users to view 5 most recently completed projects on the home page. We created the necessary controllers, models, views, and database entities required for creating, editing, and viewing projects.

Individual Contributions

Arina:

Arina implemented user story 3 in the HomeController, allowing both types of users to see the list of 5 most recently completed projects, ordered by completion time in descending order. In an asynchronous action method called Index, a nullable integer is initialized to store the ID of the person visiting the page. It is null for anonymous guests. If the visitor has successfully logged into the application, Claims securely retrieves the email address of from their identity cookie. If a matching user is found in the UserAccounts table, their unique ID is saved to the currentUserId variable.

An example of the code includes:

```
int? currentUserId = null;

if (User.Identity != null && User.Identity.IsAuthenticated)

{

    var email = User.FindFirstValue(ClaimTypes.Email);



    if (!string.IsNullOrWhiteSpace(email))

    {

        var user = await _context.UserAccounts.FirstOrDefaultAsync(u => u.Email ==
email);
```

```
        if (user != null)
        {
            currentUserId = user.Id;
        }
    }
}
```

Then a query retrieves all the projects from the database whose status is marked as "complete". It orders the projects by ID in descending order and displays only the most recent 5 entries to satisfy the sprint requirements. Example of a query for fetching the list of 5 most recently completed projects visible to all students:

```
var completedProjects = await _context.Projects
    .Where(p => p.Status == "complete")
    .OrderByDescending(p => p.CompletedAt)
    .ThenByDescending(p => p.Id)
    .Take(5)
    .ToListAsync();
```

The system uses Claims for two primary purposes: to stay consistent with the ProjectsController that Magdalena wrote and to ensure a secure way to identify the user. User's ID is passed to the front-end via ViewBag return View(completedProjects), and the list of 5 projects is sent to Index.cshtml to be rendered on the home page.

Arina also added the link to the latest version of Bootstrap to Layout.cshtml and adjusted the code in Login.cshtml and Register.cshtml for the elements to align properly on both mobile devices and laptops.

An example of the code includes:

```
<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-12 col-md-6 col-lg-4">
```

Lastly, Arina added a regular expression to the Project entity and ViewModels to ensure that users can not use numbers in the author's field.

Magdalena:

Magdalena worked on user story 1. She ensured that the Bootstrap framework was integrated for rendering all pages of the web application. Specifying the screen size within the div tags, she achieved the proper alignment of page elements when they are viewed on a small screen.

Example of the code fragment from Projects/Create.cshtml:

```
<div class="row justify-content-center">

    <div class="col-12 col-lg-8 col-xl-7">

        <div class="card shadow-sm border-0">

            <div class="card-body p-4 p-md-5">

                <div class="mb-4">

                    <h1 class="h2 fw-bold mb-2">Add Project</h1>
```

For a smoother user experience, Magdalena added a back button on most of the pages to help the user get back to the home page more intuitively.

Magdalena also successfully implemented user story 2. In the ProjectsController, she adds the [Authorize] attribute at the top to allow only logged-in users to add new projects. She uses User.FindFirstValue(ClaimTypes.Email) for getting the logged-in user's email from their security claims. The controller includes AppDbContext to be able to do "CRUD" (Create, Read, Update) operations on the Projects table. It also implemented a private static helper method, IsCompleteStatus, to handle case-insensitive string comparisons. Therefore, the system accurately identifies completed projects and manages the CompletedAt timestamps consistently. The Index method fetches all projects that belong to the logged-in user and sorts them by ID in descending order. There is also a method called details built for retrieving and displaying the project details, and it uses GET.

Magdalena has a GET Create method for displaying an empty form for a new project and a POST Create method that validates the input data via ModelState.IsValid. If the input is valid, it maps the CreateProjectViewModel data to a new Project entity, assigns the current UserAccountId, and saves it to the database. If there are issues with input

validation, an error message is displayed. She also adds the [ValidateAntiForgeryToken] attribute for security and prevention of cross-site request forgery attacks.

Although the edit and delete projects functionality wasn't part of the sprint, Magdalena also added them because it made sense. Similarly, she used the GET Edit method to find a specific project by ID, but only if it belongs to the logged-in user. It prevents unauthorized users from messing with the projects of others. The POST Edit method verifies that the project still belongs to the user, updates all the properties with the new user input, and saves the changes. She uses try and catch for proper error handling. The delete method allows a user to remove only their own projects. If the project is deleted successfully, a SuccessMessage is displayed via TempData.

Magdalena created a Project entity, ensuring that all the elements are required and have a set [MaxLength()] to protect the integrity. She also added CreateProjectViewModel.cs that contains a title, description, author, programming language, and status, along with the validation and error messages for these elements.

Example of the code related to status:

```
[Required(ErrorMessage = "Status is required.")]

[RegularExpression("^(pending|in-progress|complete)$", ErrorMessage = "Please select a valid status.")]

public string Status { get; set; } = string.Empty;
```

Finally, Magdalena created secure and responsive forms for users in Create.cshtml and Edit.cshtml views with Bootstrap 5.3. She uses Bootstrap grid classes like col-12 col-lg-8 to properly align and make elements readable on both mobile devices and desktop monitors without horizontal scrolling. The asp-validation-summary displays high-level model errors returned from the controller. Individual asp-validation-for spans provide immediate feedback for specific fields. She included ValidationScriptsPartial for jQuery-based validation, which provides a faster user experience. The Create.cshtml includes a "Save Project" submit button and a "Cancel" link that safely redirects the user back to the home page.

Acceptance Tests

Test 1 – Responsive pages on mobile devices

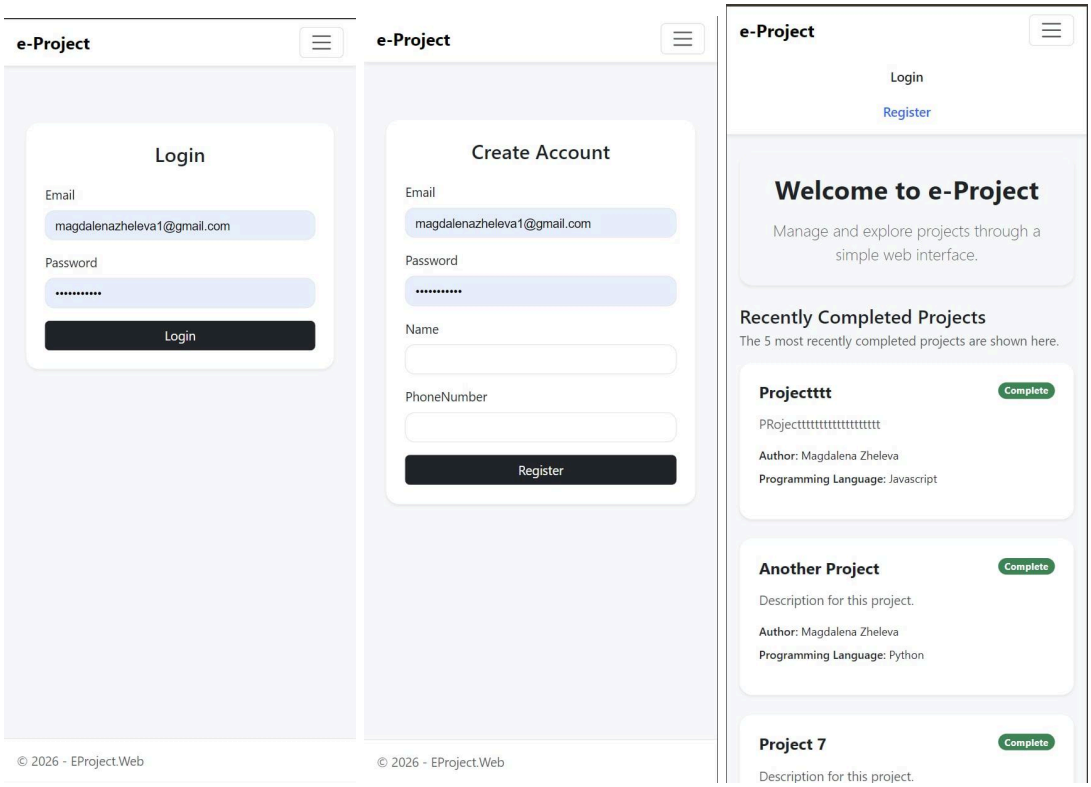
User Story: As a user, I want to see all pages of the web app being responsive and visualize well on mobile devices.

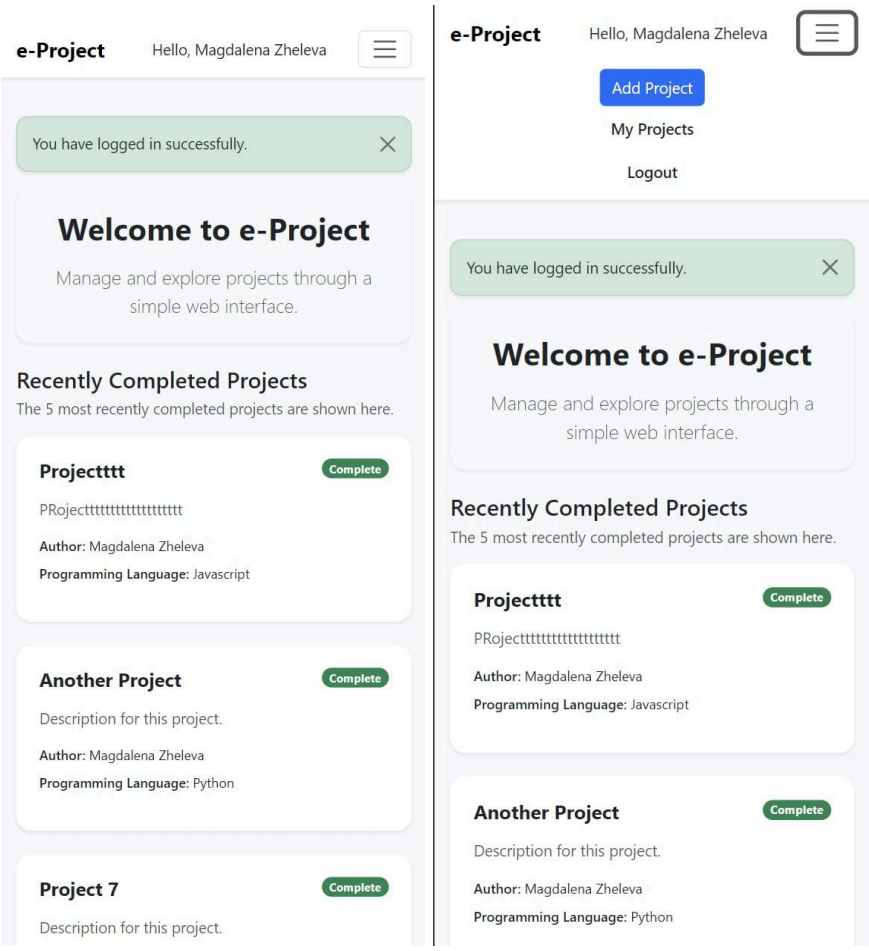
Input: User opens the web application on a mobile device and clicks through the Home, Login, Register, and Project pages.

Expected Output:

- Page elements should be properly aligned, and horizontal scrolling should not appear
- Existing menu links should be located in the top navigation menu and displayed on all pages
- Login, Logout, Register, and other links should be visible according to the current visitor's logged-in status

Result: There is no horizontal scrolling, and the top navigation menu is displayed in all pages that should be visible to anonymous and logged-in users.





Test 2 – Responsive pages on PC

User Story: As a user, I want to see all pages of the web app being responsive and visualize well on PC.

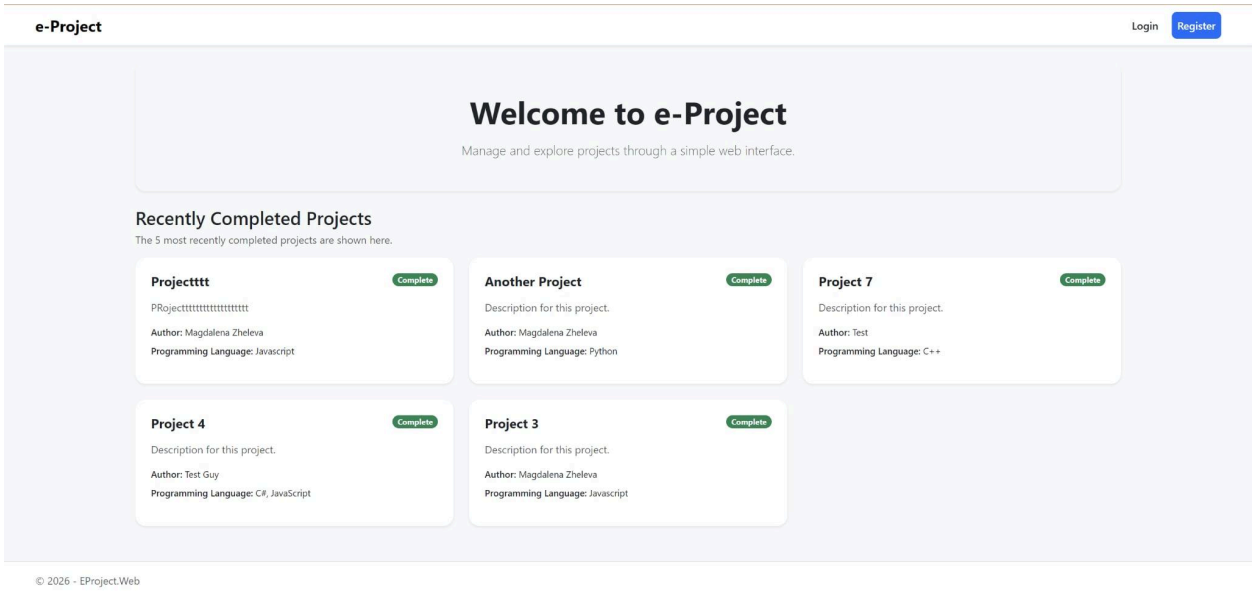
Input: User opens the web application on a desktop browser and navigates through the Home, Login, Register, and Project pages.

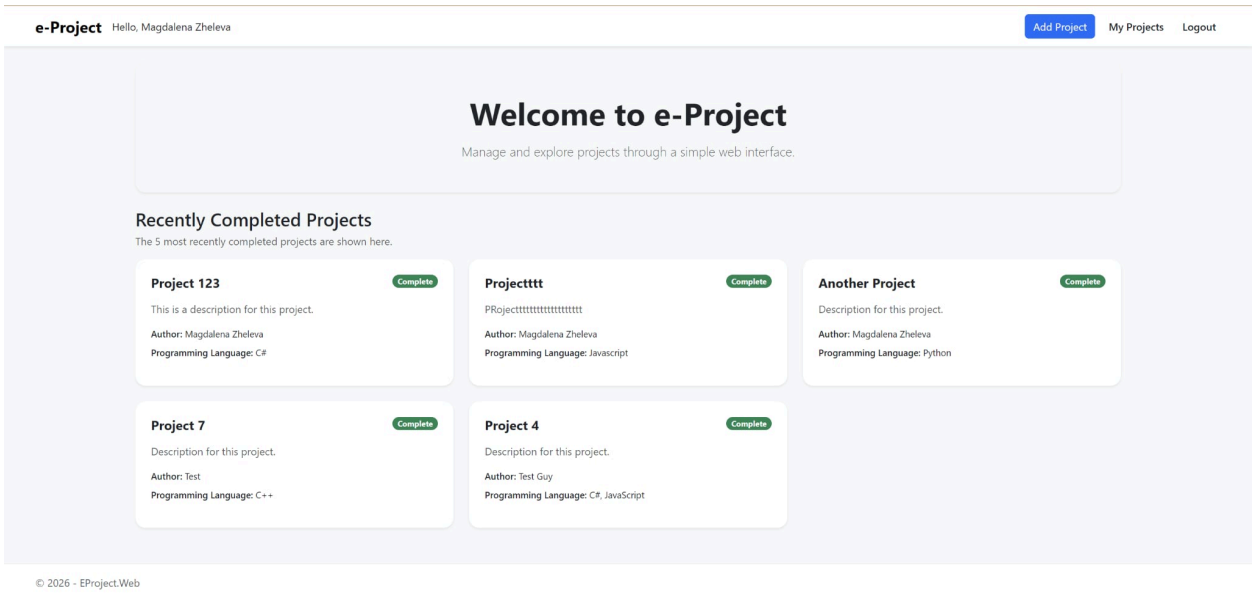
Expected Output

- Page elements should be properly aligned in a wide-screen format via the Bootstrap grid system.
- The top navigation menu should be visible on all pages.

- Login, Logout, and Register links should be visible according to the visitor's authentication status.

Result: the application renders the desktop layout, and the navigation menu is persistent across all views, depending on whether the user is logged in.





Test 3 – Adding a project successfully

User Story: As a logged-in user, I want to be able to add projects to the e-Project application.

Input:

Title: Project

Description: Testing new project

Author: John Doe

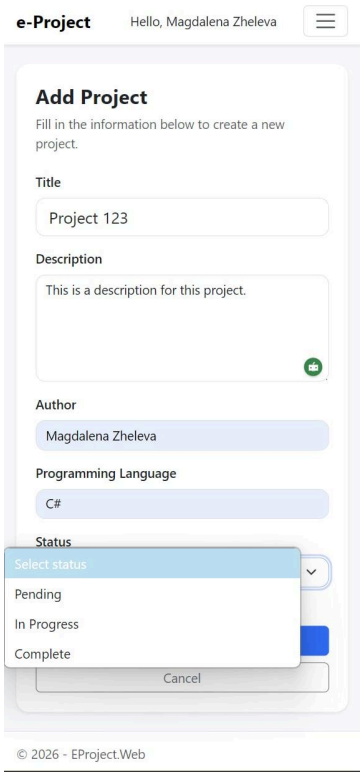
Programming language: Python

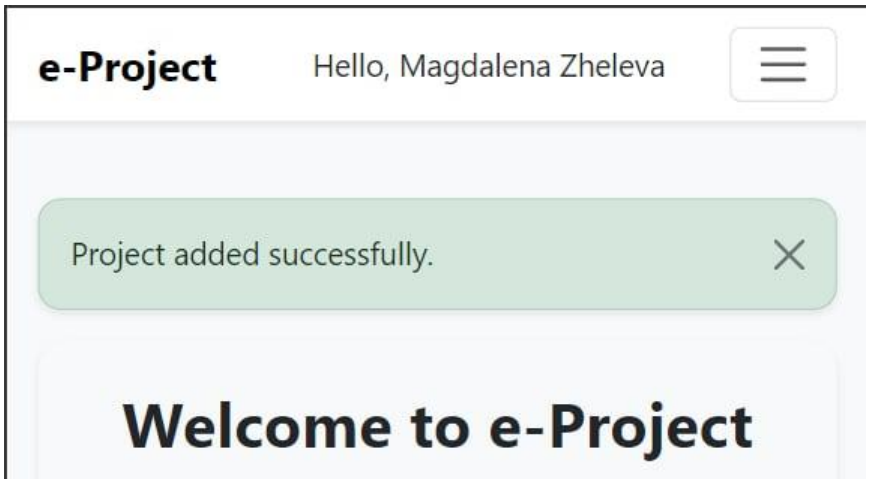
Status: in-progress

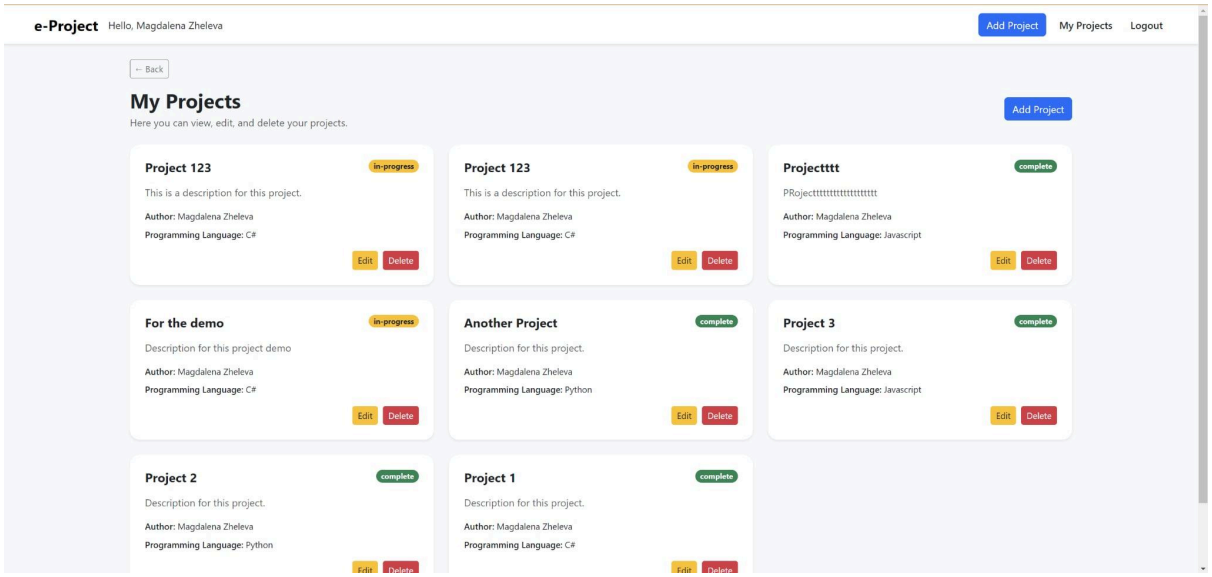
Expected Output:

- Logged-in users see an extra item in the main menu – “Add Project,” which redirects them to the project entry page
- A project form displays all the required fields, including a title, description, author, programming language, and status (pending, in-progress, complete)
- The project entry page validates all of the above properties and saves the entry in the database
- After a successful entry, the user is redirected to the home screen

Result: The project is successfully added to the database, and a success message is displayed.







Test 4 – Missing info fails validation in the add project form

User Story: As a registered user, I forget to fill in some fields or provide long responses.

Input:

Title: (Leave empty)

Author: (Leave empty)

Description: (Leave empty)

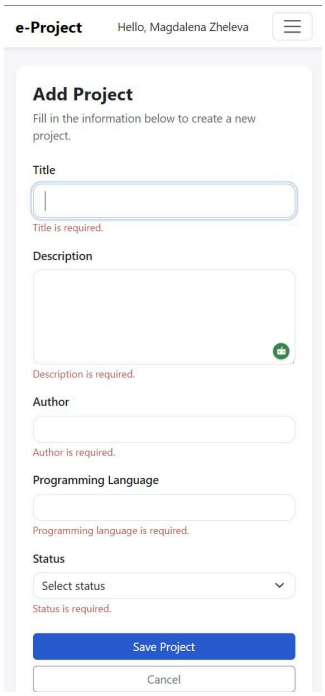
Programming Language: (Leave empty)

Status: (Do not select an option / keep "Select status")

Expected Output:

- All errors (validation, application, etc) should be properly reported to the user

Result: The application shows errors on the same page without saving the data, prompting the user to re-enter.



Test 5 – Missing info fails validation in the add project form

User Story: As a registered user, I fill in insufficiently short sentences in the fields.

Input:

Title: I

Description: a

Author: m

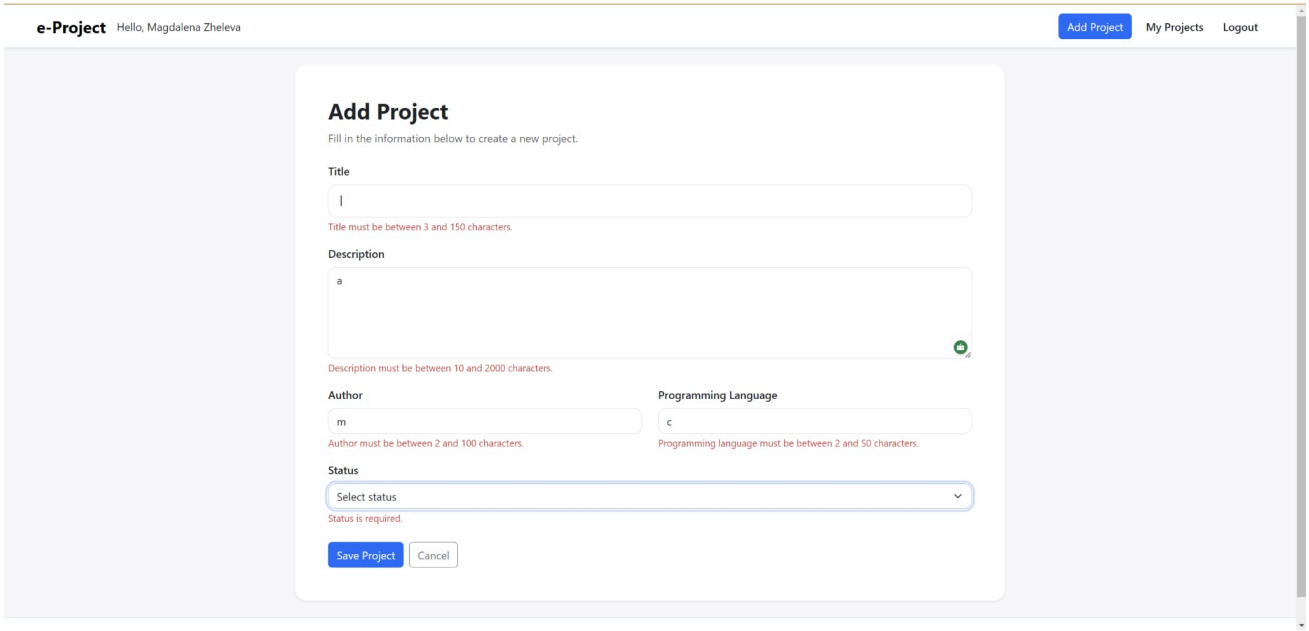
Programming Language: c

Status: (Do not select an option / keep "Select status")

Expected Output:

- Errors related to length should be properly reported to the user

Result: The application shows errors on the same page without saving the data, prompting the user to re-enter.



Test 6 – 5 most-recently completed projects

User Story: As a user, I want to see a list of the 5 most-recently completed projects.

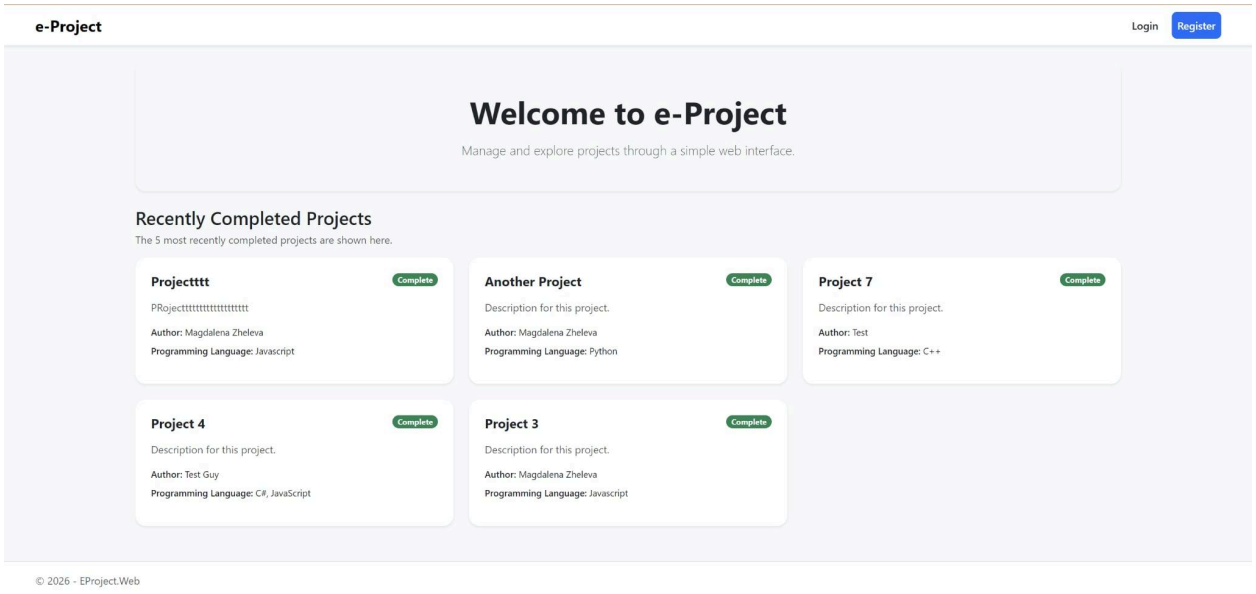
Input: User views the list of completed projects on the home page.

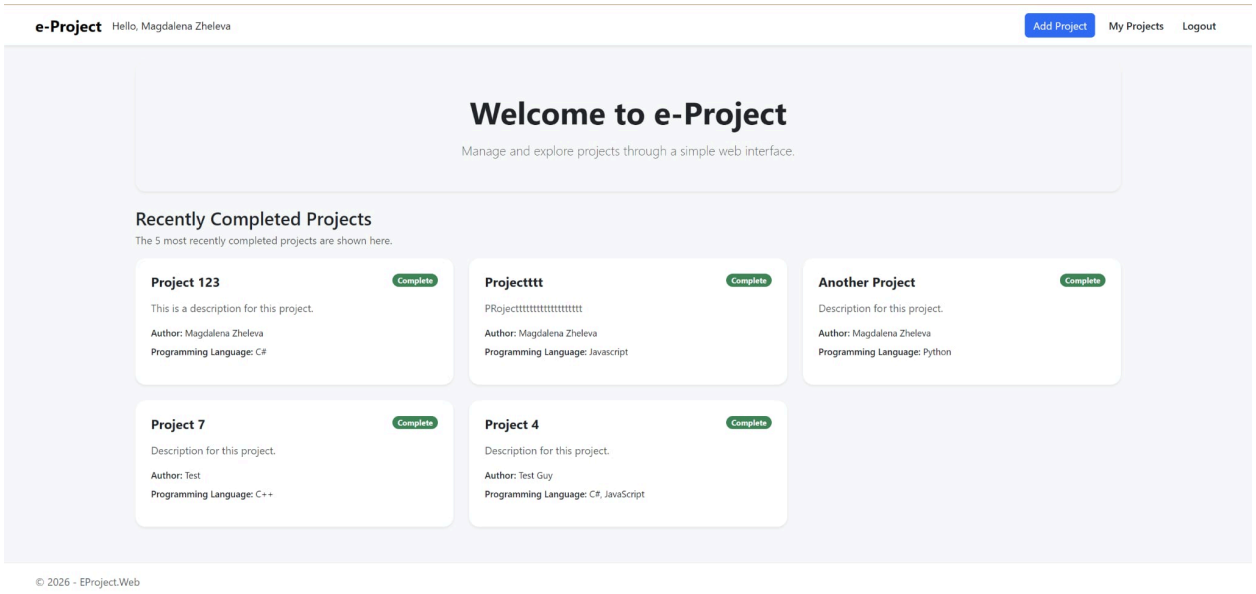
Expected Output:

- The home screen displays a list of 5 most-recently completed projects, in addition to the existing welcome message

- Both types of users see the list, ordered by completion time in descending order (newest one at the top)

Result: Any user sees the 5 most-recently completed projects ordered by completion time in descending order.





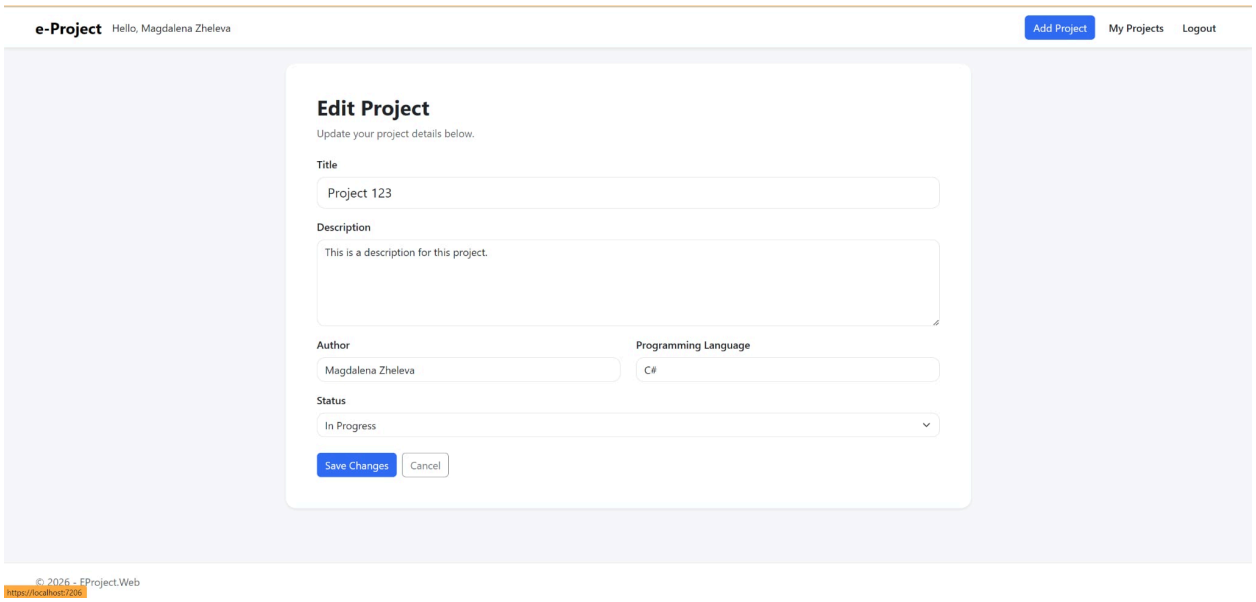
Test 7: Update Project Details (Edit) - additional functionality

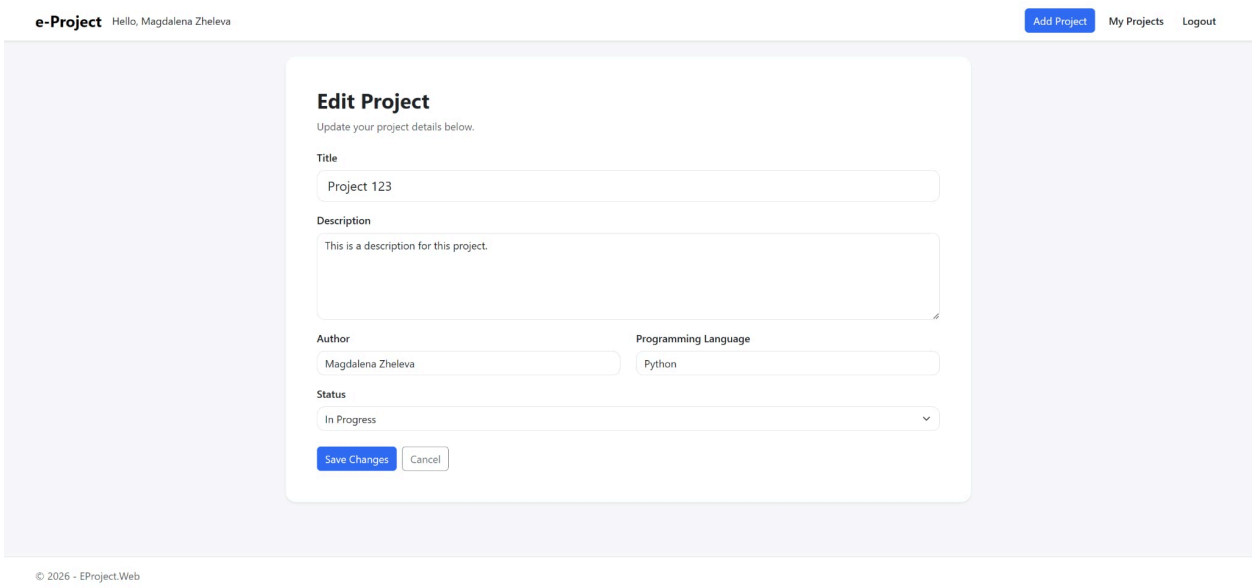
User story: A logged-in user changes the programming language of their project.

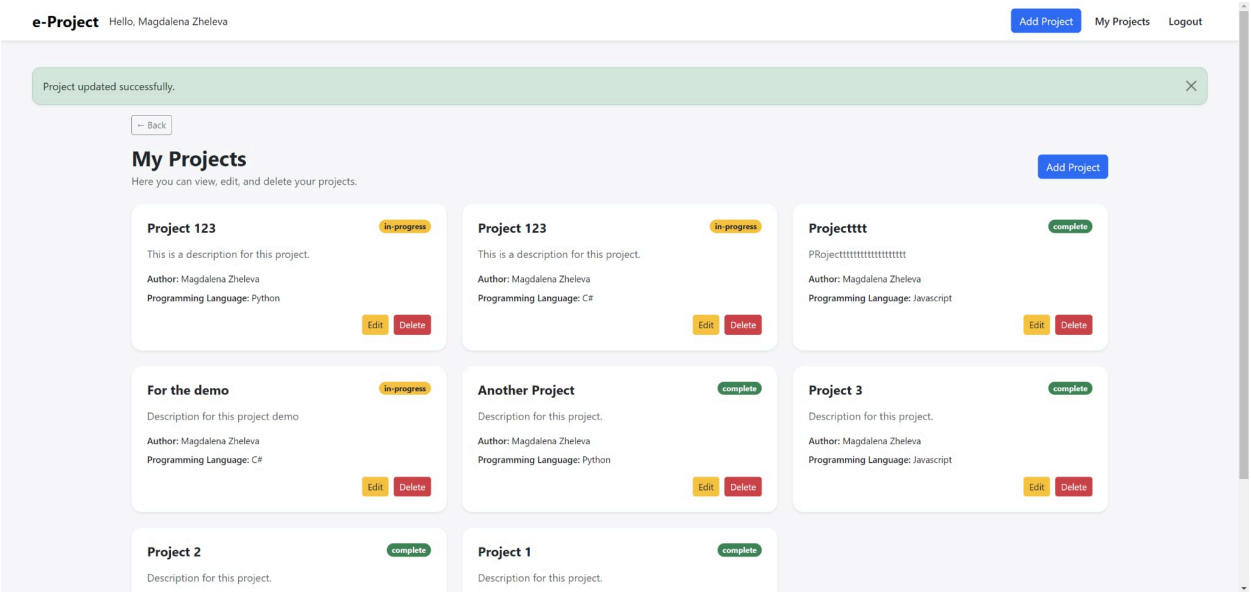
Input: Programming language: Python

Expected Output: The text should switch from C# to Python in the programming language field.

Result: the updates have been saved in the database and are visible to the user, and the success message is displayed on top.







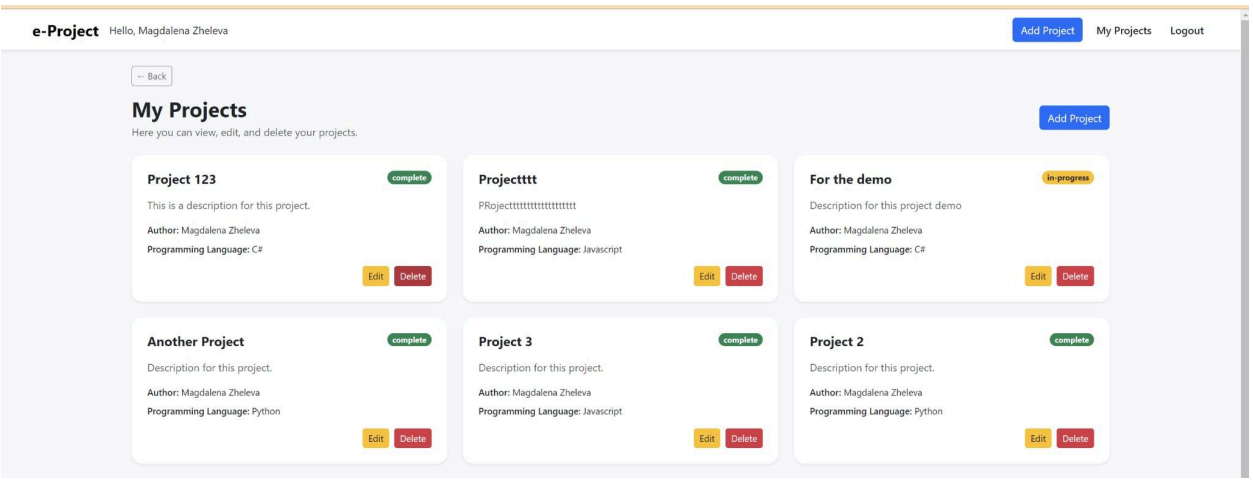
Test 8: Delete a project - additional functionality

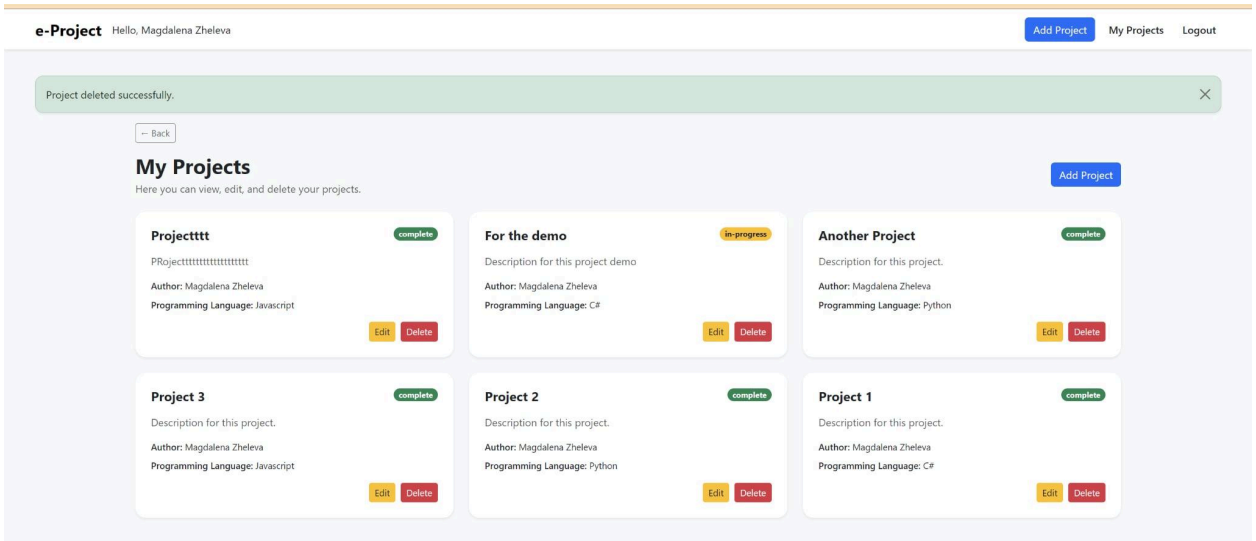
User story: A logged-in user deletes a chosen project.

Input: A user presses a delete button.

Expected Output: The project gets deleted from the database and is no longer visible to the user.

Result: the project gets deleted and is no longer visible to the user.





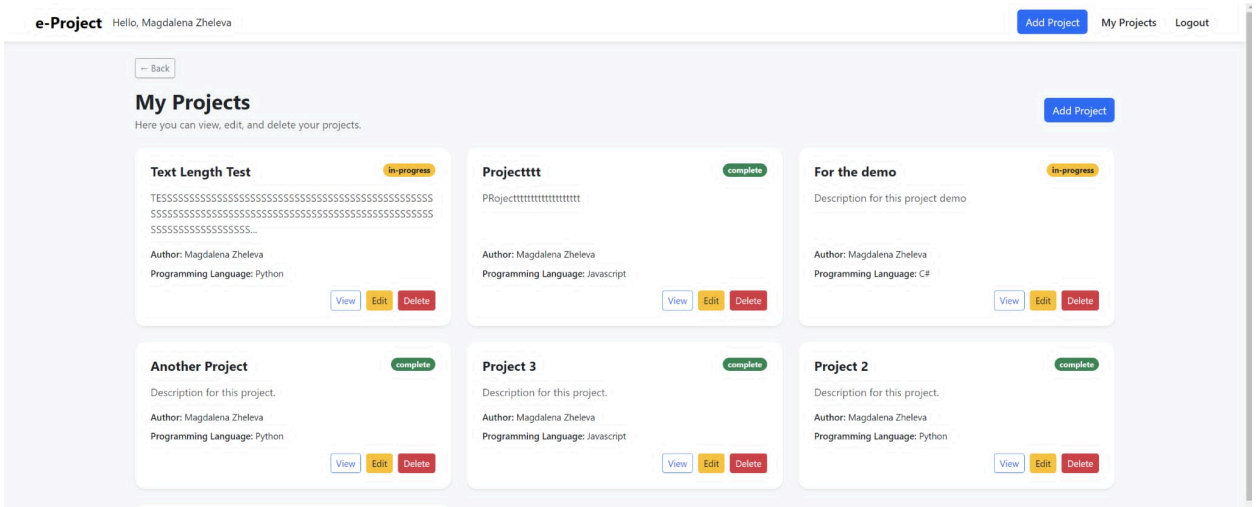
Test 9: View project details - additional functionality

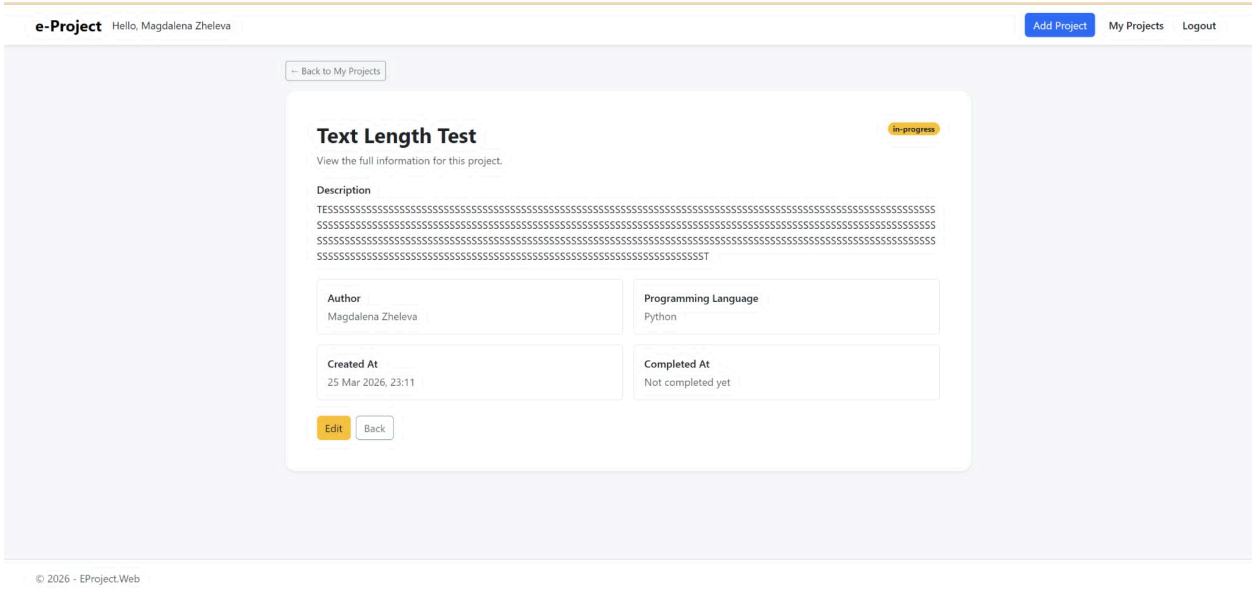
User story: A logged-in user wants to view project details.

Input: A user presses a view button.

Expected Output: The project details are shown on a new page.

Result: The project details are shown on a new page.


e-Project
Hello, Magdalena Zheleva
Add Project
My Projects
Logout
← Back
My Projects
Here you can view, edit, and delete your projects.
Add Project
Text Length Test
in-progress
TESSSS


e-Project
Hello, Magdalena Zheleva
Add Project
My Projects
Logout
← Back to My Projects
Text Length Test
in-progress
View the full information for this project.
Description
TESSSS